AI Potholes Detection

Safer roads through faster visual inspection

A Linux/Jupyter computer-vision prototype for detecting visible potholes in road images and video

Team: [Your name / team name]
Platform: LinuxONE-ready Python and Jupyter workflow

---

1. The problem

Potholes cause vehicle damage, accidents, traffic disruption, and avoidable maintenance costs. Manual road inspection is slow, inconsistent, and difficult to scale across a city.

Road authorities need an evidence-based way to screen large volumes of road imagery and prioritize field verification.

Design principle: assist inspectors with fast, reviewable detections; do not replace engineering judgment.

---

2. The solution

A one-class YOLO object detector identifies visible potholes and returns:

- Bounding boxes around candidate potholes
- Confidence scores for review and thresholding
- Annotated images or video frames as evidence
- Validation metrics: precision, recall, mAP50, and mAP50-95

The project includes a reproducible Linux/Jupyter notebook, dataset preparation script, inference examples, and deployment notes.

---

3. How it works

1. Download official RDD2022 India road-damage data.
2. Convert Pascal VOC XML annotations to YOLO format.
3. Keep the RDD2022 D40 pothole class as a single target class.
4. Fine-tune a pretrained YOLO model.
5. Validate on a held-out split.
6. Run image, video, or webcam inference and export evidence.

The workflow is intentionally transparent so a team can retrain it with local road images.

---

4. LinuxONE value and impact

LinuxONE provides an enterprise Linux environment for a secure, repeatable, centrally operated inspection workload.

A future service can ingest survey-vehicle or smartphone frames, run inference in batches, and send detections to a maintenance or GIS workflow. Centralized operations, security, scalability, and auditability matter when several inspection teams or cities use the same pipeline.

Beneficiaries: public-works departments, transport agencies, maintenance contractors, drivers, and pedestrians.

Success measures: local recall, precision, false positives per kilometer/frame, processing speed, and time from capture to maintenance-ready report.

---

5. Prototype, limitations, and next step

Prototype: Python 3.10+, JupyterLab, OpenCV, Ultralytics YOLO, official RDD2022 data, and an automated XML-to-YOLO converter.

Limitations: image detection does not measure physical depth, repair cost, or road ownership. Night, rain, glare, camera angle, and local road styles can reduce accuracy.

Next step: add representative local imagery, calibrate thresholds, connect detections to GIS/work-order systems, and evaluate on a geographically held-out test set before operational use.

Open-source resources: RDD2022 project, Pothole-600 project page, Ultralytics documentation, OpenCV, and Jupyter. See the repository README for URLs and attribution.